`Process` message, it splits the text into words, counts them, and sends the result back to the master.

Listing 8.12 Worker implementation

```
import akka.actor.typed.{ ActorRef, Behavior }
import akka.actor.typed.scaladsl.Behaviors
import akka.actor.typed.receptionist.{ Receptionist, ServiceKey }

object Worker {

  val RegistrationKey = ServiceKey[Worker.Command]("Worker")

  sealed trait Command                                                    ❶
  final case class Process(text: String, replyTo: ActorRef[Master.Event]) ❶
      extends Command                                                     ❶
      with CborSerializable                                               ❷

  def apply() = Behaviors.setup[Command] { context =>
    context.log.debug(
      s"${context.self} subscribing to $RegistrationKey")
    context.system.receptionist ! Receptionist                            ❸
      .Register(RegistrationKey, context.self)

    Behaviors.receiveMessage {
      case Process(text, replyTo) =>
        context.log.debug(s"processing $text")
        replyTo ! Master.CountedWords(processTask(text))                  ❹
        Behaviors.same
    }
  }

  private def processTask(text: String): Map[String, Int] = ...
}
```

❶ Protocol

❷ Serialization

❸ Registration at initialization

❹ Processes the message

With this, you have a complete view of the application. The next section shows how to start it.